

Шпаргалка по Git и командной разработке

1. Что такое Git и зачем он нужен

Git — система контроля версий. Она хранит историю изменений проекта: кто, когда и что менял. Позволяет:

- откатиться к любой прошлой версии;
- вести разработку нескольких людей параллельно;
- объединять чужие изменения со своими;
- понимать, кто и зачем внёс правку.

GitHub / GitLab / Bitbucket — это сервисы (хостинги), где хранится удалённая копия репозитория. Git работает и без них, локально.

2. Три «зоны» в Git — это главное для понимания

Каждый файл проходит путь через три области:



Зона	Что это	Как туда попасть
Рабочая директория	Файлы, которые вы реально редактируете	Просто меняете файл
Индекс (staging)	«Корзина» с изменениями, готовыми к коммиту	git add
Репозиторий	Зафиксированная история (коммиты)	git commit

Отдельно есть **удалённый репозиторий** (remote) — на сервере. Туда отправляют через **git push**.

3. Состояния файлов



Состояние	Что значит	Видно в git status как
Untracked	Git ещё не следит за файлом (новый)	Untracked files
Modified	Файл изменён, но не добавлен в индекс	Changes not staged for commit

Состояние	Что значит	Видно в <code>git status</code> как
Staged	Файл добавлен в индекс, готов к коммиту	<code>Changes to be committed</code>
Unmodified / Committed	Файл сохранён в истории, изменений нет	не показывается

Команда, которую смотрят постоянно:

```
git status      # текущее состояние всех файлов
```

4. Начало работы

```
git init          # создать новый репозиторий в текущей папке
git clone <url>   # скопировать существующий репозиторий с сервера
```

Первичная настройка (один раз на компьютере):

```
git config --global user.name "Имя Фамилия"
git config --global user.email "you@example.com"
git config --global init.defaultBranch main # ветка по умолчанию
```

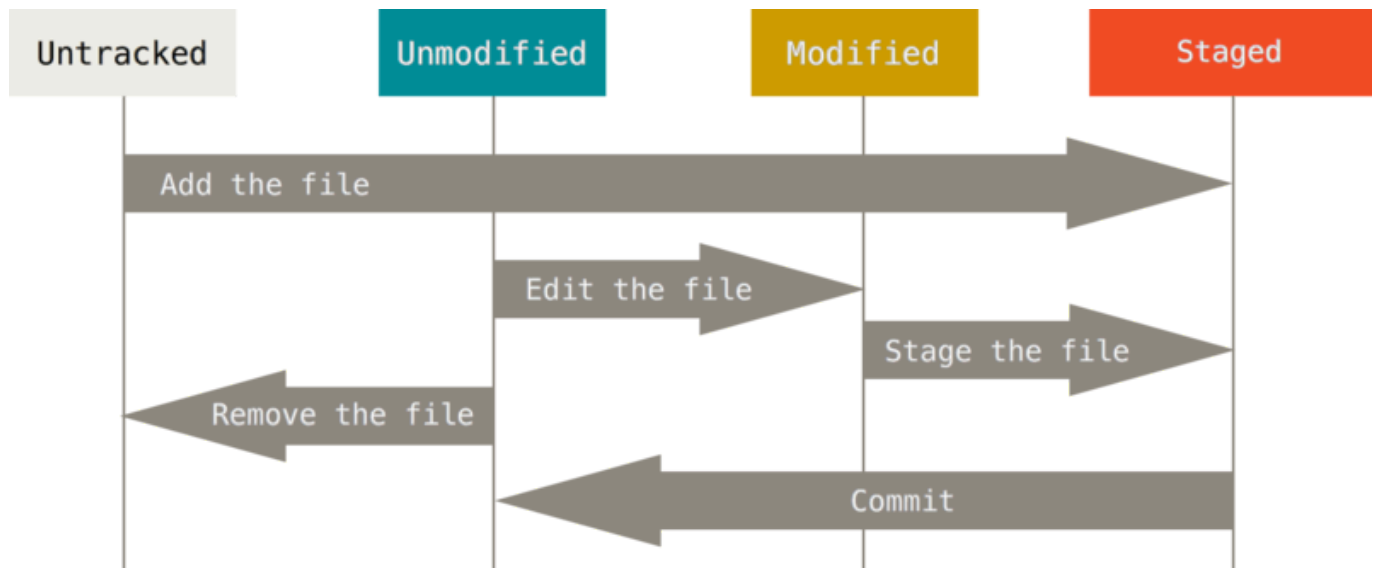
5. Базовый цикл работы (самое частое)

```
git status        # что изменилось
git add <файл>    # добавить конкретный файл в индекс
git add .          # добавить все изменения
git commit -m "Описание" # зафиксировать изменения в историю
git push          # отправить на сервер
git pull          # забрать изменения с сервера
```

Когда что:

- `git add` — «я хочу включить эти правки в следующий коммит».
- `git commit` — «фиксирую снимок состояния с описанием».
- `git push` — «делюсь своими коммитами с командой».
- `git pull` — «беру чужие коммиты к себе» (= `git fetch` + `git merge`).

Правило хорошего тона: коммит — это одно логически законченное изменение. Сообщение пишут понятно: что и зачем («Добавил валидацию формы логина»), а не «фиксы» / «апдейт».



6. Просмотр истории и изменений

```
git log                # история коммитов
git log --oneline --graph # компактно, с графом веток
git diff               # что изменено, но НЕ добавлено в индекс
git diff --staged      # что добавлено в индекс (попадёт в коммит)
git show <hash>         # детали конкретного коммита
git blame <файл>        # кто и в каком коммите менял каждую строку
```

7. Ветки (branches) — основа командной работы

Ветка — независимая линия разработки. Позволяет делать новую функцию, не ломая основной код.

```
git branch              # список веток
git branch <имя>        # создать ветку
git switch <имя>        # переключиться на ветку (современный способ)
git switch -c <имя>     # создать и сразу переключиться
git checkout <имя>      # старый способ переключения
git branch -d <имя>     # удалить ветку (после слияния)
git branch -D <имя>    # удалить принудительно
```

Типичный сценарий командной разработки:

```
git switch main
git pull                # обновили main
git switch -c feature/login # своя ветка под задачу
# ... работаем, коммитим ...
git push -u origin feature/login # отправили ветку на сервер
# затем создаём Pull Request / Merge Request
```

Почему не работают сразу в main: main должна оставаться рабочей и стабильной. Все правки — через отдельные ветки и ревью.

8. Слияние веток: merge и rebase

Merge — объединяет ветки, сохраняя историю

```
git switch main
git merge feature/login      # влить feature/login в main
```

Создаёт «коммит слияния». История ветвистая, но честная.

Rebase — переносит коммиты поверх другой ветки

```
git switch feature/login
git rebase main              # переписать свои коммиты поверх свежего main
```

История становится линейной и чистой. **Важно:** не делайте rebase веток, которые уже видны другим (переписывает историю).

	merge	rebase
История	ветвистая, с коммитами слияния	линейная, чистая
Безопасность	безопасно всегда	только для своих локальных веток
Когда	вливать готовую фичу в main	актуализировать свою ветку перед PR

9. Конфликты слияния

Конфликт возникает, когда двое поменяли одни и те же строки. Git помечает их в файле:

```
<<<<<< HEAD
мой вариант
=====
чужой вариант
>>>>>> feature/login
```

Как решать:

1. Открыть файл, выбрать/объединить нужный вариант, убрать маркеры <<<, ==, >>>.
2. `git add <файл>` — пометить как решённый.
3. `git commit` (для merge) или `git rebase --continue` (для rebase).

Отменить процесс: `git merge --abort` или `git rebase --abort`.

10. Отмена изменений (спасательный круг)

Задача	Команда
Убрать файл из индекса (оставить правки)	<code>git restore --staged <файл></code>
Откатить правки в файле к последнему коммиту	<code>git restore <файл></code>
Изменить последний коммит (сообщение/файлы)	<code>git commit --amend</code>
Отменить коммит, создав обратный	<code>git revert <hash></code>
Сбросить ветку на коммит (опасно)	<code>git reset --hard <hash></code>
Мягкий сброс (правки остаются)	<code>git reset --soft <hash></code>

revert — безопасно (для общих веток): создаёт новый коммит-отмену. **reset --hard** — переписывает историю и **удаляет** изменения. Только локально и осознанно!

11. Временно отложить работу — stash

Когда нужно срочно переключиться, но коммитить рано:

```
git stash          # спрятать незакоммиченные изменения
git stash list     # список «заначек»
git stash pop      # вернуть последние изменения и удалить из stash
git stash apply    # вернуть, но оставить в stash
```

12. Работа с удалённым репозиторием

```
git remote -v      # список удалённых репозиторий
git remote add origin <url> # привязать сервер
git push -u origin <ветка>  # первый push ветки (запоминает связь)
git fetch          # скачать изменения, НЕ сливая
git pull           # скачать + слить
```

- **fetch** — просто узнать, что появилось нового (безопасно).
- **pull** — узнать и сразу влить в текущую ветку.

13. .gitignore — что НЕ хранить в Git

Файл **.gitignore** в корне перечисляет, что игнорировать:

```
node_modules/  
*.log  
.env  
dist/  
.idea/
```

Не храните: зависимости, секреты/пароли, временные файлы, артефакты сборки.

14. Командный процесс в двух словах (Git Flow упрощённо)

1. `git pull` — обновить main.
2. `git switch -c feature/задача` — своя ветка.
3. Работа → `git add` → `git commit` (часто, небольшими порциями).
4. `git push -u origin feature/задача`.
5. Создать **Pull Request** → код-ревью от коллег.
6. Исправить замечания → новые коммиты в ту же ветку.
7. После аппрува — **merge** в main, удалить ветку.

15. Шпаргалка-минимум (распечатать и повесить)

```
git status          # где я и что изменилось  
git add .           # добавить всё в индекс  
git commit -m "..."  
git pull            # забрать чужое  
git push            # отдать своё  
git switch -c name  # новая ветка  
git switch main     # вернуться в main  
git merge name      # слить ветку  
git log --oneline --graph # посмотреть историю
```

16. Частые ошибки новичков

- ❌ Коммитят всё подряд одним огромным коммитом → ✅ дробите по смыслу.
- ❌ Работают прямо в `main` → ✅ всегда отдельная ветка под задачу.
- ❌ Не делают `pull` перед началом → ✅ обновляйтесь, чтобы меньше конфликтов.
- ❌ `git push --force` в общую ветку → ✅ почти никогда; ломает историю команды.
- ❌ Кладут пароли/.env в репозиторий → ✅ используйте `.gitignore`.
- ❌ Сообщения коммитов «фикс», «ещё фикс» → ✅ пишите осмысленно.